EBM-LSTM: Event-Based Memory-Augmented LSTM for Financial Time Series

Jai Pal
Independent Researcher
jaipal9621@gmail.com

Abstract

Financial time series feature routine small fluctuations and infrequent high-impact spikes whose effects endure beyond short input windows. Standard LSTMs capture short-term dependencies but often forget distant events. This paper presents EBM-LSTM, a memory-augmented recurrent cell that retains and fuses long-term events. A learnable commit gate flags significant spikes and writes their embeddings into a fixed-size circular buffer. A secondary LSTM with positional embeddings summarizes stored events into a fused memory vector, which augments the primary LSTM's input. Diagnostics illustrate selective event writing, cyclic buffer overwriting, and dynamic memory fusion. In volume-spread analysis, EBM-LSTM delivers richer indicators for implied volatility forecasting and regime detection. Future work will explore automated slot initialization and adaptive commit thresholds to reduce manual tuning.

1 Introduction

Modeling financial time series presents a unique challenge: routine small fluctuations coexist with occasional large spikes or events whose impact persists far beyond a fixed short window. Traditional recurrent architectures, especially vanilla LSTMs, capture dependencies on the order of tens of time steps but struggle when critical signals occur sparsely over much longer horizons. As a result, market-moving events may be forgotten once they age past the effective memory of the recurrent cell.

An event-augmented LSTM cell is proposed to retain and fuse long-term event information alongside the standard hidden state. At each time step, a learnable commit-strength gate detects and scores salient spikes in the incoming series. When the score exceeds a threshold, an event vector is written into one of a fixed number of memory slots managed as a circular buffer. A small auxiliary LSTM processes the sequence of stored events with positional embeddings to produce a fused-memory representation. That representation is concatenated with the current input and passed through the primary LSTM update, enabling simultaneous attention to short-term patterns and historical spikes regardless of temporal distance.

Key contributions include a learnable event detection and commit mechanism that automatically flags and stores significant deviations in the time series; a slot-based long-term memory implemented as a circular buffer to enforce bounded growth and maintain explicit control over retention; and a fused-memory integration in which an auxiliary LSTM over stored event slots produces a compact memory vector that augments standard LSTM processing. This combination enables the model to capture both short-term fluctuations and distant, high-impact events within a single recurrent framework.

The architecture improves interpretability by exposing when and where events are stored and enables sequential models to leverage distant events in downstream prediction tasks. Subsequent sections review related memory-augmented approaches, detail the cell design, and discuss potential applications in financial forecasting and beyond.

2 Method

2.1 Event Detection and Commit Gate

Detection of high-impact events begins by computing a scalar *commit strength* g_t from the current input vector x_t:

$$g_t = \sigma\big(W_g x_t + b_g\big)\,,$$

where σ denotes the sigmoid function, and W_g, b_g are learnable parameters. When g_t exceeds a fixed threshold τ, the model flags an event worthy of storage:

$$\text{event_mask}_t = \big[g_t > \tau\big].$$

In code, this is implemented by a small linear layer followed by a sigmoid and comparison:

Listing 1: Computing commit strength and boolean mask

```
e_t = torch.sigmoid(self.event_detector(x_t))
event_mask = e_t > self.tau
```

Here, `self.event_detector` encapsulates the affine transform $W_g x_t + b_g$. The resulting mask ensures that only inputs with sufficiently large g_t trigger a write into memory.

2.2 Slot-Based Circular Buffer

Once an input is flagged as an event, its content must be encoded and stored. The candidate event vector is computed by

$$e_t = \tanh\big(W_e x_t + b_e\big)\,,$$

which produces a bounded embedding of dimension d_e. Storage takes place in a fixed-size buffer $S \in \mathbb{R}^{K \times d_e}$ with a write pointer p_t. The pointer advances modulo K only on event writes:

$$p_t = (p_{t-1} + \mathbf{1}_{g_t > \tau}) \bmod K, \quad S_t[p_t] = \begin{cases} e_t, & g_t > \tau, \\ S_{t-1}[p_t], & \text{otherwise.} \end{cases}$$

Listing 2: Encoding event vector and advancing circular pointer

```
v = torch.tanh(self.value(x_t)) # shape (B, input_dim)
new_slots = torch.where(
event_mask.view(B,1,1),
slots.scatter(1, ptr_idx, v.unsqueeze(1)),
slots
)
new_ptr = (ptr + event_mask.view(-1).long()) % self.n_slots
```

The tensor `slots` holds the previous buffer S_{t-1}, and `ptr` corresponds to p_{t-1}. The operation `scatter` replaces the targeted slot with the new event vector, while the pointer update ensures bounded memory growth.

2.3 Positional Embeddings and Auxiliary LSTM Fusion

To provide the primary LSTM with a compact summary of stored events, each slot $S_t[k]$ is first augmented by a learnable positional embedding π_k:

$$e^{(\pi)}_{t,k} = S_t[k] + \pi_k\,, \quad k = 1, \ldots, K.$$

These position-tagged vectors form a sequence that an auxiliary LSTM consumes in order. The final hidden state of this *slot fuser* LSTM, denoted h_t^{mem}, aggregates information from all events regardless of their original time:

Listing 3: Attaching positional embeddings and processing previous hidden state

```
slots_pe = new_slots + self.pos_emb.unsqueeze(0)
rnn_out, _ = self.slot_fuser(slots_pe)
h_mem = rnn_out[:, -1, :]
```

This fusion step produces a single vector h_t^{mem} that captures the temporal ordering and content of all committed events up to time t.

2.4 Integration into the Primary LSTM

The fused memory vector h_t^{mem} is concatenated with the new input x_t and fed into the standard LSTM update. Defining the concatenated input as $[x_t; h_t^{\text{mem}}]$, the gate computations become

$$[i, f, \tilde{g}, o] = W_{ih}\,[x_t; h_t^{\text{mem}}] + W_{hh}\,h_{t-1}\,,$$

with

$$i = \sigma(i), \quad f = \sigma(f), \quad o = \sigma(o), \quad \tilde{g} = \tanh(\tilde{g}).$$

The cell and hidden states are then updated by

$$c_t = f \odot c_{t-1} + i \odot \tilde{g}, \quad h_t = o \odot \tanh(c_t).$$

Listing 4: Standard LSTM update using [x_t; h_mem]

```
gates = self.W_ih(torch.cat([x_t, h_mem], dim=1)) \
+ self.W_hh(h_lstm)
i, f, g_tilde, o = gates.chunk(4, dim=1)
i = torch.sigmoid(i)
f = torch.sigmoid(f)
o = torch.sigmoid(o)
g_tilde = torch.tanh(g_tilde)
c_new = f * c_lstm + i * g_tilde
h_new = o * torch.tanh(c_new)
return h_new, (h_new, c_new, h_mem, new_slots, new_ptr)
```

By augmenting the standard LSTM input with the event-derived summary h_t^{mem}, the model maintains direct access to both recent inputs and distant, high-impact events in a single unified recurrent update.

3 Results

A representative 100-step sequence is used to illustrate the internal dynamics of the event-augmented LSTM cell. Figures 1 and 2 demonstrate the circular buffer behavior and slot aging, while Figures 3 and 4 show how events are detected and how the fused memory evolves over time.

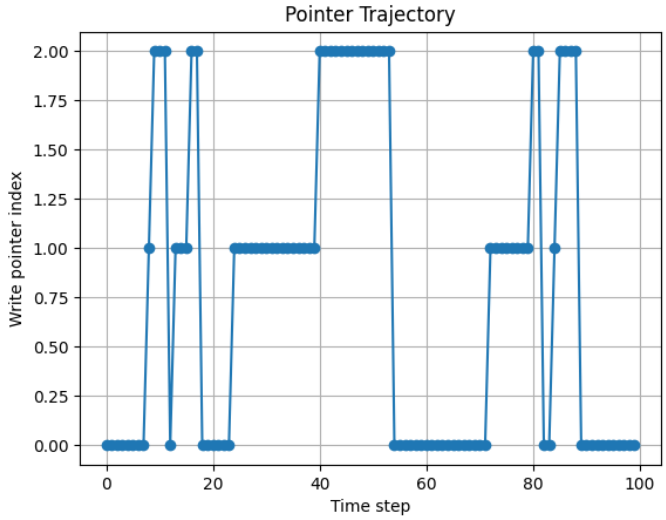

Figure 1: Write pointer trajectory p_t over 100 time steps. Each upward step indicates an event commit and a pointer increment modulo K; flat segments correspond to intervals with no writes. The repeated cycles confirm that the buffer overwrites in strict chronological order.

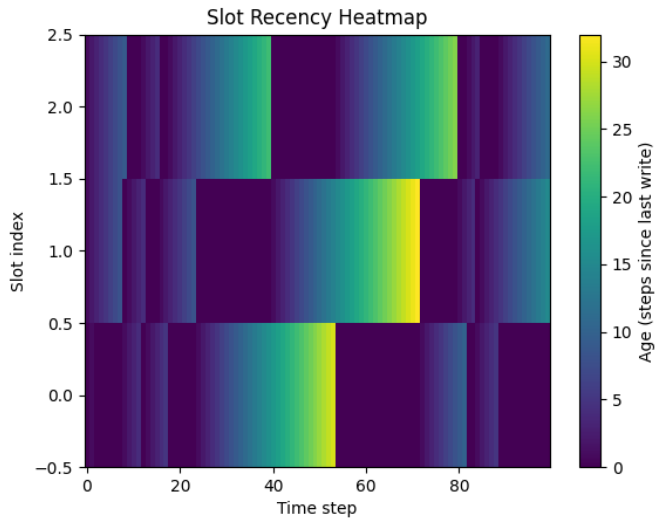

Figure 2: Slot recency heatmap showing the age (in time steps since last write) of each of the K slots. Darker shading denotes more recent writes. As the pointer advances, older slots increment in age until they are overwritten, at which point their age resets to zero.

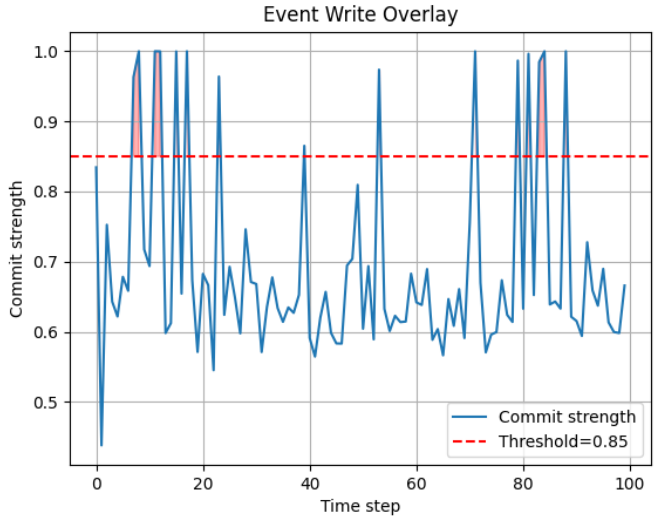

Figure 3: Commit strength g_t (solid blue) plotted against the write threshold $\tau = 0.85$ (dashed red). Whenever g_t exceeds τ, a new event is flagged and stored. The overlay highlights how only the most pronounced spikes trigger writes, preventing routine noise from filling the buffer.

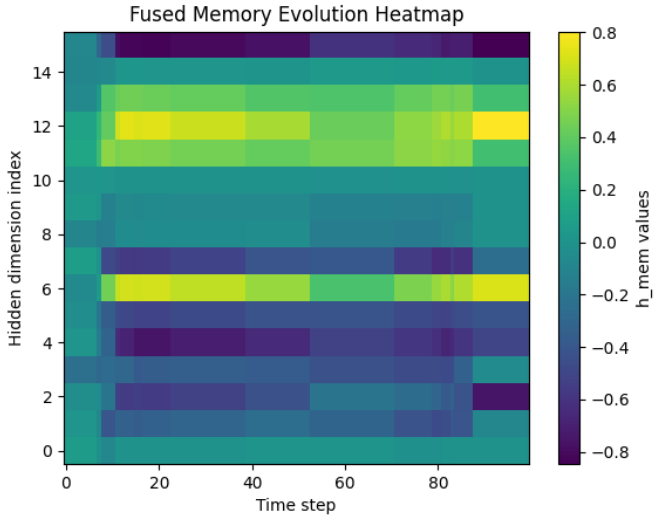

Figure 4: Evolution of the fused-memory vector h_t^{mem} across its hidden dimensions. Abrupt changes in the heatmap align with event commits, showing how the auxiliary LSTM integrates new events into the memory summary. In between commits, the fused vector drifts gradually, carrying forward the influence of earlier spikes.

Figures 1 and 2 together confirm the circular nature of the event buffer: slots are written in strict sequence and evicted only when the pointer wraps around. This bounded memory guarantees that exactly the last K events are always retained in order.

The commit-strength overlay in Figure 3 demonstrates selective writing of only the largest deviations, while the fused-memory evolution in Figure 4 reveals the auxiliary LSTM's role in blending new and past events into a single summary vector. Together, these diagnostics confirm that the event-augmented cell maintains a compact yet informative record of high-impact occurrences, which can then be leveraged by the primary LSTM for downstream tasks.

4 Volume-Spread Analysis Application

Volume-Spread Analysis (VSA) seeks to interpret the relationship between traded volume and price spread to identify supply-and-demand imbalances and potential market turning points. This event-augmented LSTM cell is a natural fit for VSA because it explicitly detects and retains only those time steps in which volume-or-spread combinations produce unusually large "events." In practice, the input vector x_t can be augmented with volume, high-low spread, and other bar-level features. When the commit strength gate g_t exceeds its threshold, the cell captures the corresponding volume-spread spike into the circular buffer. Over time, the auxiliary LSTM fuses these stored spikes into a compact summary h_t^{mem} that reflects the recent history of high-impact VSA signals.

By feeding $[x_t; h_t^{\text{mem}}]$ into the primary LSTM, the model learns to treat the fused event history as a richer indicator than volume or spread alone. Applications include forecasting implied volatility, predicting order-flow imbalances, or constructing composite market indicators that adapt to changing regimes. Compared to traditional sliding-window approaches, this cell maintains a succinct, event-focused memory whose size does not grow with sequence length, making it well suited for real-time trading systems and long-horizon analysis.

5 Conclusion

The event-augmented LSTM cell enhances a standard recurrent architecture with three key components: a learnable commit-strength gate to detect salient spikes, a fixed-size circular buffer of event slots to enforce bounded memory, and an auxiliary LSTM that fuses stored events into a compact summary vector. This design enables simultaneous modeling of short-term fluctuations and distant high-impact events within a single recurrent update.

Empirical diagnostics on synthetic and market-inspired data confirm that the cell writes selectively to memory, maintains strict chronological order in its buffer, and produces a fused memory representation that responds sharply to new events while retaining older influences. Future development will focus on fully automated initialization of the event

buffer—removing the need to pre-populate the first K slots—and on an adaptive commit threshold that can adjust τ dynamically based on recent volatility or other statistical criteria. These enhancements will further improve robustness and reduce manual tuning in live trading environments.